

Neural Cellular Automata

From self-organizing growth to a generative model

Seung Ho Go

CS236 follow-up seminar

June 28, 2026

Table of Contents

- 1 Motivation & cellular automata
- 2 Growing NCA
- 3 From mechanism to generative model: VNCA
- 4 NCA vs. Diffusion
- 5 Limitations & the paradigm map

Motivation: morphogenesis

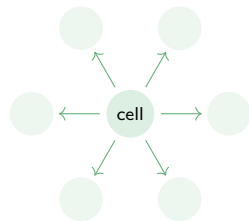
A single fertilized cell becomes a full organism.

No cell sees the whole body. Each one only:

- reads the chemical state of its **neighbors**,
- updates itself by a **shared local rule**.

Global form emerges from **local interactions** — and damaged tissue **regrows**.

Question: can we *learn* such a local rule that grows a target shape?



each cell talks only
to its neighbors

Cellular automata 101 (for newcomers)

Conway's Game of Life — the classic CA:

- a grid of cells, each alive or dead;
- every cell updates **in parallel** from its 8 neighbors;
- one **fixed, hand-crafted** rule (B3/S23) for *all* cells.

Complex patterns (gliders, “guns”) emerge from this tiny rule.

Gosper's glider gun — Lucas Vieira, Wikimedia Commons, CC BY-SA 3.0

The NCA idea (next slide)

Keep the grid + parallel local updates + one shared rule.

Replace the hand-crafted rule with a small neural network — and make it **differentiable** so we can *train* it.

From CA to *Neural* CA

Game of Life

- state $\in \{0, 1\}$
- rule: fixed lookup (B3/S23)
- not differentiable
- nothing to learn

Neural CA

- state $\in \mathbb{R}^{16}$ per cell
- rule: a tiny shared MLP
- fully differentiable
- **trained by gradient descent**

Inductive biases we get *for free*:

locality (neighbors only) · **parallelism** · **weight sharing** (same rule everywhere)

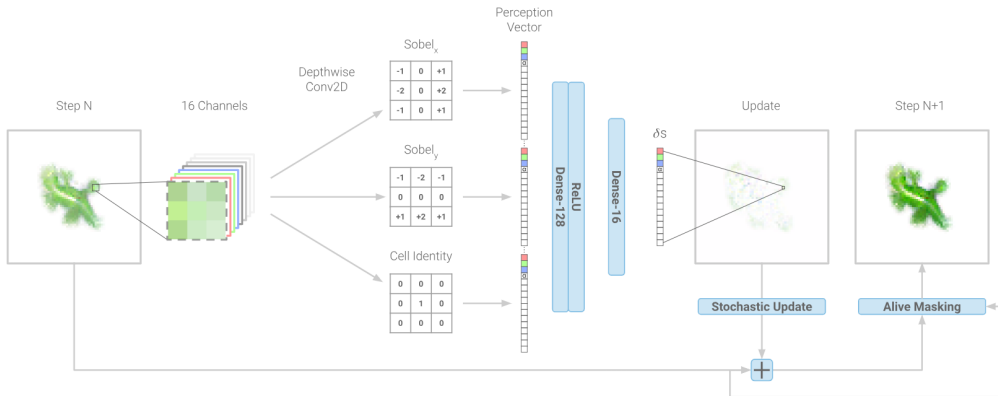
The cell state: a 16-channel grid

Grid tensor $x \in \mathbb{R}^{B \times 16 \times H \times W}$. Each cell carries 16 numbers:

- **ch 0** — **visible** grayscale intensity (what we render)
- **ch 1** — **alpha / alive** (is this cell “grown”?)
- **ch 2–15** — **hidden** channels (free scratch space the rule learns to use)

The hidden channels are the trick: they let cells store and pass *messages* (e.g. “I am the left edge of a 3”) that never appear in the final image.

One NCA step = four stages



A single update step of the model.

"Growing Neural Cellular Automata", Mordvintsev et al., Distill 2020 (CC BY 4.0)

Every cell, in parallel, with **shared weights**: **(1)** perception $16 \rightarrow 48 \rightarrow$ **(2)** update net $48 \rightarrow 128 \rightarrow 16 (\Delta x) \rightarrow$ **(3)** stochastic update (prob. 0.5) $\rightarrow$ **(4)** alive masking. $x \leftarrow \text{alive}(x + m \odot \Delta x)$, $m \sim \text{Bernoulli}(0.5)$.

Stage 1 — perception

Each cell must estimate its **local environment**. We do this with *fixed* depthwise filters on every channel:

- **identity** — the cell's own state
- **Sobel_x, Sobel_y** — gradients = “how do my neighbors differ from me?”
 $16 \text{ channels} \times \{\text{id}, \partial_x, \partial_y\} = 48 \text{ perception channels}$

These filters are **not learned** — just a fixed, cheap way to read the neighborhood. All the intelligence lives in the next stage.

Stages 2–4 — rule, asynchrony, life

2. Update net (the learned rule)

Two 1×1 convs ($48 \rightarrow 128 \rightarrow 16$) = a small MLP shared by all cells. **Last layer is zero-initialized** $\Rightarrow \Delta x = 0$ at start: the grid begins still, then learns to move (stable bootstrap).

3. Stochastic update

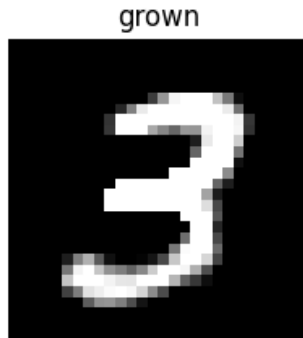
Each cell fires with prob. 0.5 $\Rightarrow$ **no global clock**; robust to update order = asynchrony.

4. Alive masking

A cell stays alive only if a neighbor has $\alpha > 0.1$. Empty space stays empty; the pattern grows outward from living tissue.

Training Growing NCA

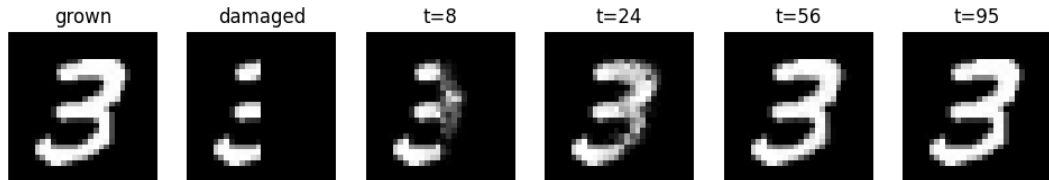
- 1 **Seed:** one center cell with $\alpha = 1$, all else 0.
- 2 **Roll out** $T \sim \mathcal{U}(64, 96)$ steps.
- 3 **Loss:** MSE between visible channel and *one* target digit.
- 4 **Sample pool:** restart from past outputs $\Rightarrow$ stable long-term growth.
- 5 **Damage (Phase 2):** erase parts of pooled states during training $\Rightarrow$ the rule **learns to repair**.



seed $\rightarrow$ rolled out $\rightarrow$ a "3"

*Note: this is reconstruction of a single image — the model does **not** learn a distribution (yet).*

The money shot: damage & regeneration



Grow a "3" → **erase half** → keep running the same rule → it **regrows** to the correct shape.

Why does it regenerate?

Nothing in the loss says “be robust.” Regeneration is a **free consequence of locality**:

- the rule only ever learned a *local* “what should I look like given my neighbors?”;
- after damage, surviving cells are an out-of-place context;
- the same local rule drives them back toward a consistent global shape.

Damage is just another starting state the local rule knows how to fix.

Act II — the pivot

Growing NCA memorizes **one** image.

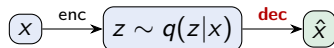
Where is the **distribution**? How do we get **new** samples?

A VAE's whole job is to learn $p(x)$ and sample from it. What if we keep the VAE machinery but make the **decoder** an NCA?

One-slide VAE recap

- **Encoder** $q(z|x) = \mathcal{N}(\mu, \sigma^2)$
- **Sample** $z = \mu + \sigma \odot \epsilon$ (reparam.)
- **Decoder** $p(x|z)$
- **Train:** maximize the ELBO

$$\mathcal{L} = \underbrace{\mathbb{E}_q[\log p(x|z)]}_{\text{reconstruction}} - \underbrace{\text{KL}(q(z|x) \parallel \mathcal{N}(0, I))}_{\text{regularizer}}$$



We swap the **decoder** box for an NCA.

VNCA = VAE with an NCA decoder

- ① **Encoder** (small CNN): $x \rightarrow \mu, \log \sigma^2 \rightarrow z \in \mathbb{R}^{16}$
- ② **Seed from** z : broadcast z to every cell of the grid (instead of a single-pixel seed)
- ③ **Decode**: run the **same NCA rule** for T steps (alive-masking off)
- ④ **Output**: visible channel $\rightarrow$ Bernoulli logits (binarized MNIST)

$$\mathcal{L}_{\text{VNCA}} = \underbrace{\text{BCE}(\hat{x}, x)}_{\text{recon}} + \beta \cdot \text{KL}(q(z|x) \parallel \mathcal{N}(0, I))$$

Growing NCA and VNCA **share the update-net architecture and the rollout loop**. Only two things change: the **seed** (z , not one pixel) and the **objective** (ELBO, not one-image MSE).

Two practical fixes (so it actually works)

β warm-up

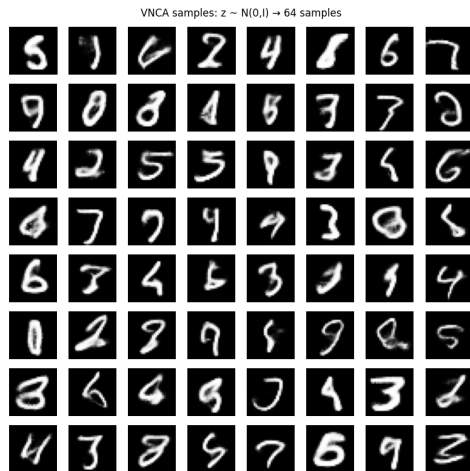
The NCA decoder is powerful $\Rightarrow$ risk of **posterior collapse** (latent ignored). Ramp $\beta : 0 \rightarrow 1$ over the first 30% of training so z stays meaningful (needed for interpolation).

Positional encoding

Broadcasting the *same* z to all cells $\Rightarrow$ cells can't tell where they are $\Rightarrow$ scribbly, mispositioned strokes. Fix: overwrite the last 2 channels each step with a fixed (x, y) coordinate map.

Plus longer training (VAEs need many epochs). These three turned noisy blobs into legible digits.

VNCA samples: $z \sim \mathcal{N}(0, I)$



Sample z from the prior $\rightarrow$ grow $\rightarrow$ **diverse digits**. A genuine generative model: it learned $p(x)$, not one image.

Latent interpolation: $3 \rightarrow 8$



Encode a “3” and an “8”, linearly interpolate z , decode each $\rightarrow$ a **smooth morph**. A continuous, meaningful latent space = the signature of a **true generative model** (and proof z did *not* collapse).

NCA vs. Diffusion: two flavors of iterative refinement

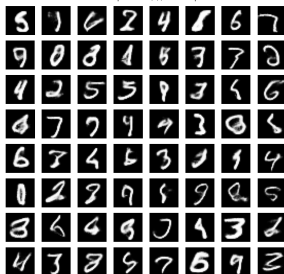
Axis	Diffusion	NCA (Growing / VNCA)
Big picture	both: iterative refinement	both: iterative refinement
Refinement	global denoising	local message-passing
What is learned	score / noise ϵ	a local update rule
Start point	pure Gaussian noise	seed (Growing) / z (VNCA)
Noise process	explicit forward SDE	none (self-organization)
Receptive field	global every step	local, grows per step
Parameters	large	tiny ($\sim$tens of thousands)
Regeneration	not built in	native (from locality)

*Both refine over many steps. The difference is **global denoising vs. local self-organization**.*

Head to head: samples & compute (MNIST)

VNCA (ours)

VNCA samples: $z \sim N(0, I) \rightarrow 64$ samples



Diffusion (ours)



	VNCA	Diffusion
Training (A100, our run)	<1 h	≈3–4 h
Parameters	~tens of thousands	U-Net (millions)
Sharpness (by eye)	comparable	comparable
Class-ambiguous samples	a few more	fewer

Comparable sharpness at a fraction of the params/wall-clock — but VNCA has a few more hard-to-read digits. (Timings approximate.)

The numbers — and a unifying view

Compactness

- NCA update net: $\sim$ **10k–20k** params
- Diffusion U-Nets: **millions–billions**

Compute is at test time

- generation = run the rule for T steps
- more steps $\rightarrow$ more refinement
- a **test-time compute** knob

One step = message passing

An NCA step is exactly a round of **message passing on a grid graph** (a GNN with a 3×3 neighborhood and shared weights).

Diffusion / flows / NCA all belong to the same family: *iterative refinement of a sample*.

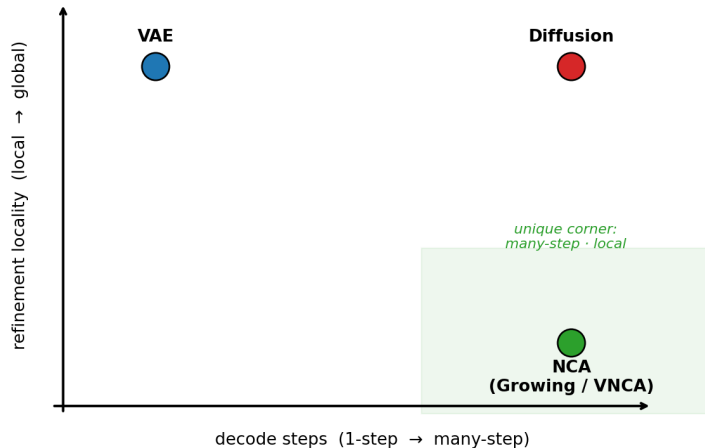
Limitations (name them before Q&A does)

- **Sample quality** below VAEs/diffusion — VNCA samples are blurry/unstable (the few unreadable digits you saw).
- **Does not scale** — high-res / color is expensive per step and training gets unstable.
- **Long BPTT** — 30–90 unrolled steps $\Rightarrow$ tricky gradient dynamics.
- **Weak on quantitative metrics** (e.g. FID).

*These are the price of living in the **many-step** · **local** corner $\rightarrow$ next slide.*

The generative-paradigm map

Generative paradigms: iterative refinement, global vs local



x: decode steps (1 $\rightarrow$ many)

y: refinement locality (local $\rightarrow$ global)

- VAE: 1-step, global
- Diffusion: many-step, global
- NCA: **many-step, local**

NCA occupies a **unique corner**. Its position *is* its trade-off: regeneration, robustness, compactness — at the cost of sample quality and scale.

Takeaways

- ① NCA = a **learnable, local** update rule on a grid — a differentiable cellular automaton.
- ② **Regeneration** is not a feature; it is a direct consequence of **locality**.
- ③ Swapping a VAE decoder for an NCA promotes the mechanism into a **true generative model** (VNCA).
- ④ NCA and Diffusion are both iterative refinement — **local self-organization vs. global denoising**.
- ⑤ NCA lives in the **many-step · local** corner: compact & robust, but not (yet) high-fidelity.

Thank you — questions?